

Forrajeo multi-agente con memoria externa en grillas: una historia experimental

Jere Figo

Proyecto final de Aprendizaje por Refuerzo

Resumen

Este informe resume una serie de experimentos de aprendizaje por refuerzo multi-agente en un entorno de forrajeo. El objetivo era entrenar hormigas locales que encontraran comida, la llevaran al hub y, eventualmente, usaran una memoria externa discreta para coordinarse. La historia experimental no fue lineal. El principal progreso no vino de “más comunicación” por sí sola, sino de definir correctamente la tarea, hacer que la señal de crédito fuera aprendible, aumentar la cobertura con más agentes y cambiar la arquitectura del crítico centralizado. En 25x25 se obtuvo una política que resuelve el gate de 23 entregas bajo movimiento muestreado. En 50x50 la mejora más importante coincide con el paso de un crítico MLP plano a un crítico espacial `strided_cnn`; dentro de esa familia se llegó a resultados fuertes pero todavía confundidos con más hormigas, selección de checkpoints, escritura compartida y penalizaciones de escritura. En 250x250 apareció una lección negativa: el retorno con shaping puede crecer aunque las entregas reales sean cero. La conclusión honesta es que el sistema aprendió forrajeo multi-agente útil y escaló mejor de lo esperado, pero todavía no demuestra causalmente comunicación emergente mediante bytes.

1. Motivación

El problema que se estudia es simple de describir y difícil de aprender: varios agentes locales deben encontrar fuentes de comida, recoger unidades de comida y volver al hub para entregarlas. Cada agente observa solo una vecindad pequeña y ejecuta una política compartida. Durante el entrenamiento se permite un crítico centralizado, pero durante el despliegue la política sigue siendo local. Esta separación corresponde al marco de entrenamiento centralizado con ejecución descentralizada, usado en algoritmos multi-agente como MAPPO y MADDPG [6, 2].

La motivación no es solo maximizar una métrica. La pregunta interesante es semántica: qué comportamiento aparece cuando se combinan búsqueda local, memoria en el ambiente y presión de entregar comida. Un resultado útil no debería ser únicamente “alto reward”. Debería mostrar ciclos completos de forrajeo, regreso al hub, uso razonable del espacio y robustez bajo mapas aleatorios. Por eso este informe distingue tres cosas que a menudo se mezclan:

- desempeño: cuánta comida se entrega, con qué tasa de éxito y con cuántos pasos;
- mecanismo: qué cambios de entorno, política o crítico hicieron posible el desempeño;
- comportamiento: si los videos muestran búsqueda, transporte y retorno, o solo movimiento con shaping.

El trabajo se ubica cerca de MAPPO [6], PPO [4], GAE [3] y aprendizaje de comunicación multi-agente como CommNet y DIAL [5, 1]. La diferencia práctica es que aquí la comunicación no es un canal diferenciable directo entre agentes. Es una memoria discreta escrita en celdas del ambiente. Eso hace que la atribución causal sea más difícil: ver bytes no-cero no implica que esos bytes estén coordinando la política.

2. Entorno y semántica

La semántica del entorno cambió de manera decisiva durante el proyecto. En la versión relevante para este informe, `food_count` no significa posiciones independientes de comida. Significa mordidas totales. `food_sources` controla cuántas fuentes concentran esas mordidas y cada fuente se agota. La recompensa cruda principal es la entrega en el hub: una unidad entregada da +1. La recogida puede tener bonus de shaping en algunas corridas, pero no es el objetivo final.

Esta distinción cambia la interpretación de todos los resultados. Un resultado de 23/23 en 25x25 no significa visitar 23 puntos aislados. Significa completar 23 entregas desde 6 fuentes agotables. Cambiar `food_sources` con `food_count` fijo cambia la geometría de la tarea, la repetición de rutas y la dificultad de redescubrir fuentes.

Tabla 1: Variables semánticas que no deben tratarse como detalles menores.

Variable	Interpretación para el informe
<code>food_count</code>	Total de unidades que pueden ser entregadas. Es el denominador natural de la fracción entregada.
<code>food_sources</code>	Número de fuentes agotables. A igual <code>food_count</code> , menos fuentes implican fuentes más densas.
<code>sampldmovement</code>	Despliegue estocástico de movimiento. Muchas políticas buenas viven en la distribución, no en el argmax.
<code>greedy movement</code>	Despliegue determinista. Es más estricto y frecuentemente empeora el retorno.
<code>critic_architecture</code>	Arquitectura del crítico centralizado. Cambia la señal de valor durante PPO aunque el actor local no reciba observación global en ejecución.
<code>write_while_moving</code>	Permite escribir mientras se mueve. Cambia el costo y la frecuencia efectiva de escritura.

3. Método

La base de entrenamiento es PPO multi-agente con política compartida, ventajas estimadas con GAE y un crítico centralizado. PPO optimiza una función objetivo recortada que evita cambios demasiado grandes de política [4]. MAPPO aplica esta idea a agentes cooperativos y suele usar un crítico centralizado para estabilizar el aprendizaje [6]. En este proyecto, el actor debe ejecutar con información local; el crítico puede ver más información durante entrenamiento.

La arquitectura del crítico es una frontera causal del proyecto:

- `mlp`: crítico plano sobre observaciones centralizadas aplanadas. Fue usado en líneas tempranas y en el 50x50 eficiente inicial.
- `strided_cnn`: crítico espacial sobre planos de grilla y rasgos de entidades. Aparece en la línea 50x50 de proximidad y full-layout.
- `set_cnn`: familia posterior usada en diagnósticos 250x250. No es una continuación limpia de los experimentos 50x50.

Esta separación es central. Si una corrida 50x50 mejora después de pasar de `mlp` a `strided_cnn`, no se puede atribuir la mejora solamente a más hormigas, más bits, fuentes más densas o escritura compartida. El crítico cambió el problema de crédito que PPO resuelve durante entrenamiento.

4. Historia experimental

4.1. Cambio de contrato de la tarea

El primer cambio grande fue conceptual. La tarea dejó de premiar fuertemente recoger comida y pasó a medir entregas al hub con fuentes agotables. Esto convirtió el problema en forrajeo real: encontrar no basta, hay que cerrar el ciclo fuente-hub. También hizo que la densidad de fuentes fuera una variable de dificultad.

Esta frontera invalida comparaciones ingenuas con corridas viejas. Un reward alto bajo pickup reward no significa lo mismo que un reward alto bajo delivery-only reward.

4.2. Fracaso inicial de escala con un solo agente

La primera línea JAX aprendía mapas pequeños, pero colapsaba al crecer. El reporte consolidado local registra que el retorno bajaba de 12,281 en 4x4 a 0,719 en 25x25 y 0,156 en 50x50. El patrón observado era compatible con agentes que exploran o cargan comida, pero no convierten eso en ciclos repetidos de entrega.

La causa más plausible no fue falta de bits. Era una mezcla de horizonte largo, recompensa escasa, cobertura insuficiente y crédito débil.

4.3. El desbloqueo 25x25

El primer resultado fuerte aparece al combinar shaping de distancia, más hormigas y más capacidad. `DISTANCE_SHAPE` ya hacía parcialmente aprendible el gate 25x25: 13,75/23 en greedy, pero solo 7/23 muestreado. `DISTANCE_CAP4` cambia a cuatro hormigas y hidden size mayor; allí la evaluación muestreada llega a 23/23 con éxito 1,0, aunque el modo greedy cae a 2,75/23.

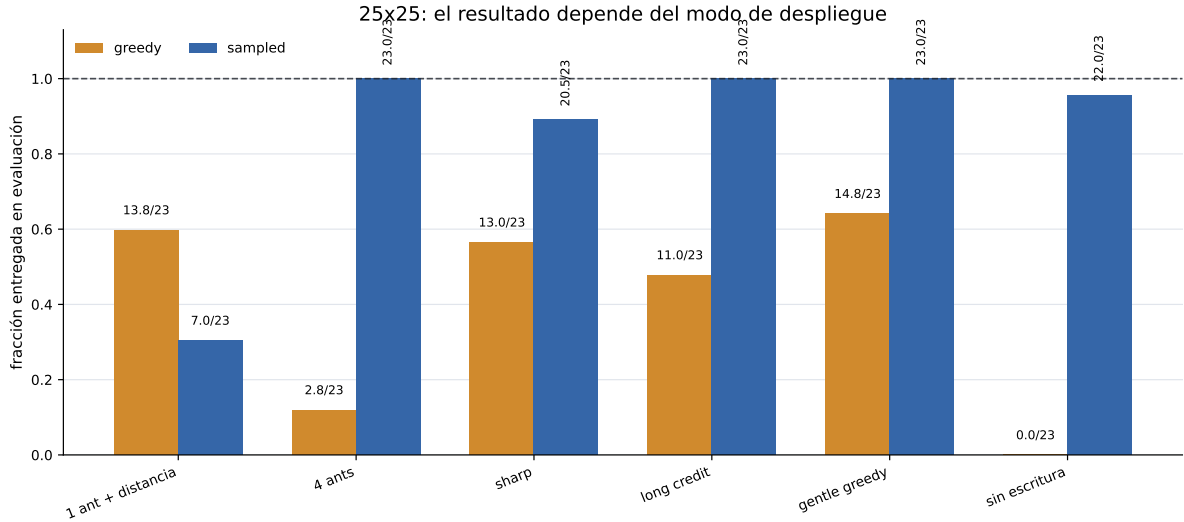


Figura 1: Evolución 25x25 bajo dos protocolos de despliegue. El resultado clave no es solo el máximo de comida entregada, sino la brecha entre movimiento greedy y movimiento muestreado.

El resultado `DISTANCE_CAP4_NO_WRITE` es una advertencia contra sobreinterpretar comunicación: incluso sin escritura, una continuación entrega 22/23 bajo movimiento muestreado. No es una ablación perfectamente controlada, porque también cambian schedule y optimizador, pero sí muestra que la solución 25x25 no requiere demostrar comunicación por bytes.

4.4. Bits contra cobertura

Los barridos tempranos de bits no muestran un salto claro de capacidad. En una línea 15x15 aproximada, el retorno de entrenamiento sube de 6,40 a 7,36 al pasar de 2 a 5 bits y luego baja levemente en 8 bits. En la línea anclada a 25x25, el retorno se mantiene bajo. En cambio, aumentar el número de hormigas en 25x25 con 3 bits produce una mejora monotonica de retorno de entrenamiento.

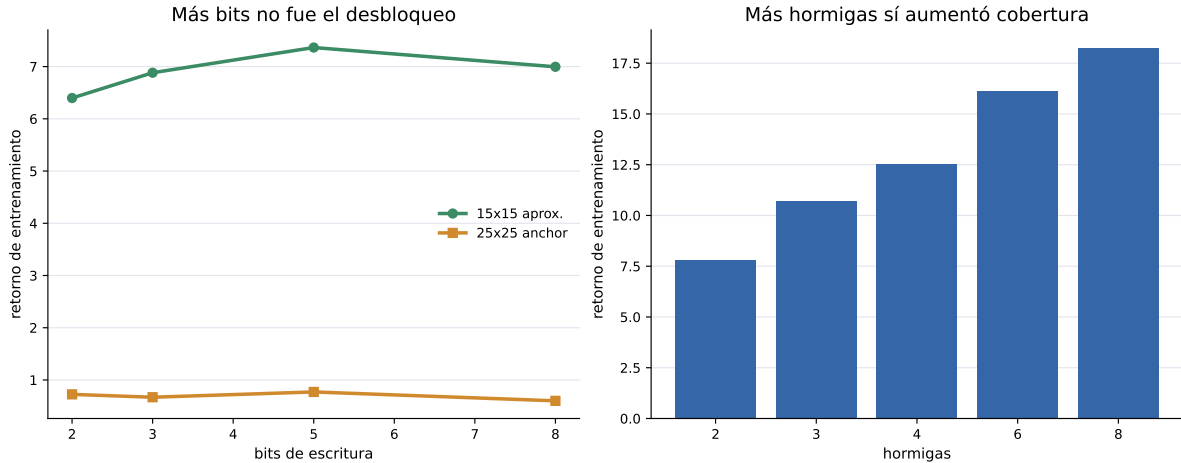


Figura 2: Los primeros barridos favorecen una interpretación simple: la cobertura multi-agente importó más que aumentar el alfabeto de escritura. Estas curvas son retornos de entrenamiento, no evaluaciones held-out.

4.5. 50x50 raro y ayudas vectoriales

Cuando la comida se vuelve rara en 50x50, el problema cambia. El agente debe descubrir fuentes con baja probabilidad y luego sostener ciclos. La línea base rara llega a 10,96/23. Agregar visión de radio 2 no ayuda. Agregar vector al hub mejora poco. Agregar vector al hub y a la comida cercana sube a 15,20/23, y selección held-out llega a 16,20/23 en la confirmación final.

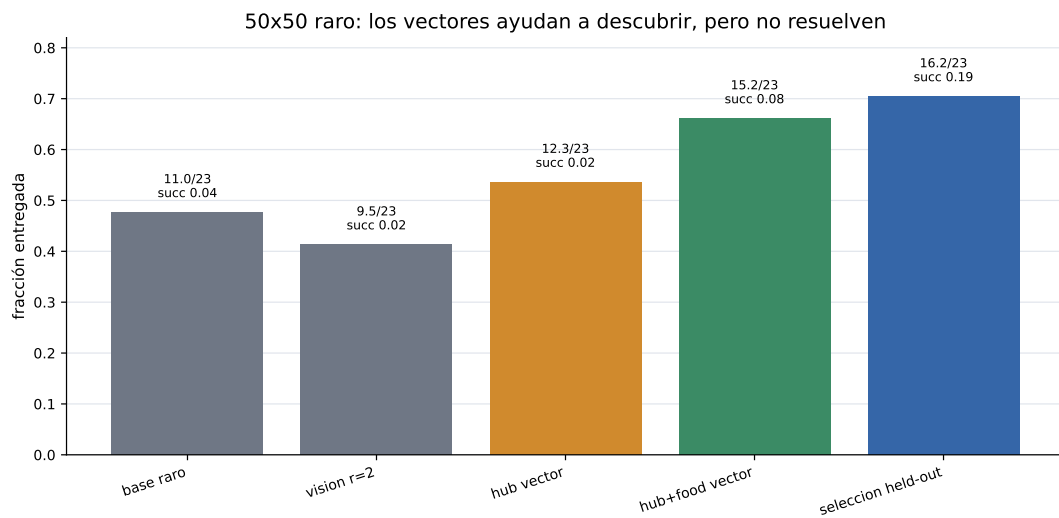


Figura 3: En 50x50 raro, las ayudas vectoriales parecen facilitar descubrimiento. La mejora no prueba comunicación emergente, porque las corridas cambian más de una variable.

4.6. La frontera del crítico en 50x50

La línea 50x50 eficiente inicial usa el crítico MLP por defecto y entrega 21,5/48. La línea posterior de proximidad y full-layout cambia a **strided_cnn**. Ese cambio es mayor: el crítico ya no procesa una observación central plana, sino una representación espacial. Dentro de esa familia aparecen mejoras importantes, pero también colapsos de checkpoint y fuerte dependencia de selección.

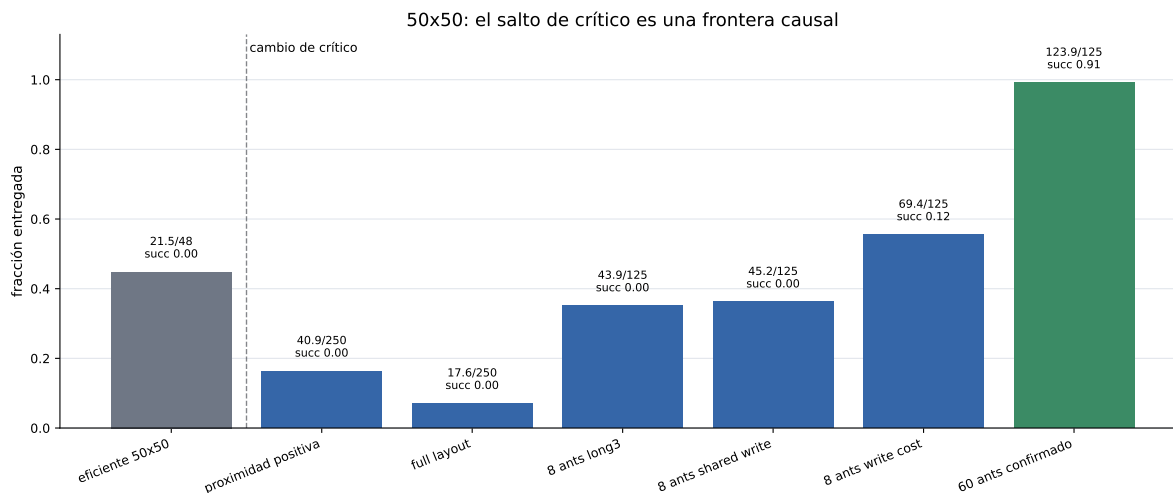


Figura 4: Resultados 50x50 seleccionados. La línea vertical marca el cambio de crítico MLP a crítico espacial. Las barras posteriores no son una sola ablación: combinan arquitectura del crítico, fuentes, ants, escritura, costos y checkpoints.

El mejor resultado local de 8 hormigas en esa familia es **shared_writes_write_cost_from_shared_best**: 69,375/125, éxito 0,125. Es progreso real, pero no una solución completa. También tiene escritura no-cero muy alta, alrededor de 0,94, por lo que no alcanza para afirmar que haya un código compacto e interpretable.

La línea reciente de 60 hormigas es la frontera actual del repo. Una evaluación de 64 episodios del checkpoint estabilizado reporta 123,90625/125, fracción 0,99125, éxito 0,90625. Esto es fuerte como ingeniería de comportamiento 50x50, pero cambia demasiadas cosas a la vez: 60 hormigas, 8 bits, actor con identidades repetidas, continuación desde checkpoint, critic **strided_cnn**, selección por evaluación, temperaturas y escritura casi saturada. Debe presentarse como resultado prometedor, no como prueba causal de comunicación.

4.7. 250x250: la trampa del shaping

La escala 250x250 mostró el fracaso más informativo. Corridas con **resnet_cnn** o **strided_cnn** en una fuente terminan con cero pickups y cero entregas. En la familia **set_cnn**, el autocurriculum de distancia puede producir retorno positivo de shaping, muchos agentes cargando y byte maps saturados, pero todavía cero entregas finales. Es exactamente el tipo de resultado que un informe debe separar de la métrica objetivo.

El cambio **fixed8-reset-boundary256** sí mejora la entrega local: el resumen final tiene 654 entregas y 869 pickups. La evaluación fresh-window del mismo linaje selecciona **reset_boundary_final** como campeón por mediana de entregas. Aun así, esto no resuelve la escala general. Es una intervención de reset y distancia dentro de una familia de crítico distinta.

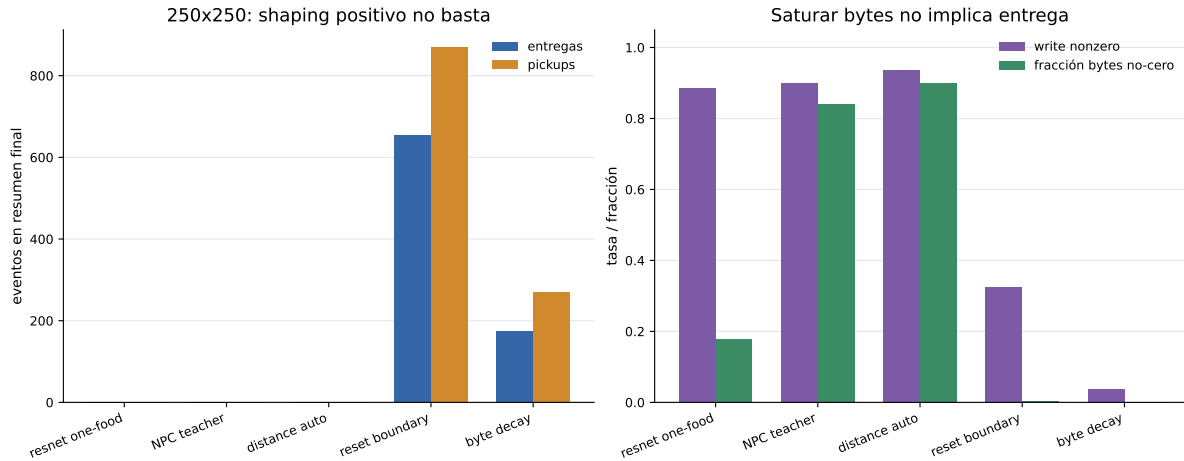


Figura 5: Diagnóstico 250x250. La primera subfigura muestra eventos reales. La segunda muestra escritura. La entrega mejora localmente con reset-boundary, mientras que saturar bytes por sí solo no implica resolver forrajeo.

5. Evolución cualitativa del comportamiento

Los videos apoyan una lectura en cuatro etapas. Primero hay movimiento local sin retorno estable. Luego aparece cobertura multi-hormiga en 25x25: las hormigas encuentran fuentes y cierran ciclos. En 50x50 MLP hay progreso parcial, pero más dispersión y menos finalización. En 50x50 con crítico espacial aparecen rutas repetidas y entregas más densas. En 250x250 se ve la diferencia entre actividad y objetivo: puede haber mucha escritura o exploración sin suficientes entregas.

6. Qué funcionó y qué no

Funcionó, con evidencia fuerte:

- redefinir la tarea alrededor de entregas reales y fuentes agotables;
- usar shaping de distancia para atravesar el horizonte largo inicial;
- aumentar cobertura con más hormigas;
- evaluar políticas muestreadas cuando el comportamiento aprendido vive en la distribución;
- cambiar el crítico centralizado a una arquitectura espacial para 50x50.

Funcionó parcialmente o con confusión causal:

- escritura compartida y penalización de escritura en 50x50;
- aumento a 8 bits, porque también cambió la escala de costo;
- línea de 60 hormigas, porque mejora mucho pero mezcla muchos factores;
- reset-boundary en 250x250, porque arregla ventanas locales pero no demuestra curriculum general.

No funcionó o no quedó demostrado:

- aumentar bits como único mecanismo de escala;

- deployment greedy como criterio único de éxito;
- interpretar retorno con shaping como entrega real;
- afirmar comunicación emergente causal sin ablaciones matched de no-write, shuffled-write y write-cost.

7. Limitaciones

La limitación principal es experimental: muchas corridas son continuaciones con varios cambios simultáneos. Por eso este informe habla de “intervenciones” y no de ablations puros salvo donde haya evidencia comparable. También hay diferencias entre checkpoint final, mejor checkpoint, selección held-out y resúmenes W&B. Esas diferencias importan porque varias corridas tienen pico temprano y luego degradación.

La segunda limitación es semántica. La memoria externa existe y se usa en el sentido de que hay acciones de escritura y bytes no-cero. Pero todavía falta demostrar que la información escrita cause mejores decisiones. Para eso hacen falta evaluaciones como: congelar política y borrar bytes, permutar bytes entre celdas, bloquear escritura solo en test, igualar costos entre 4 y 8 bits, y comparar `mlp` contra `strided_cnn` con el mismo entorno.

La tercera limitación es de escala. El resultado 60-ant 50x50 es muy bueno, pero usa muchos agentes y escritura saturada. Puede ser la dirección de ingeniería correcta, pero no responde por sí solo la pregunta de comunicación compacta.

8. Conclusión

El proyecto produjo una historia clara. Primero hubo que corregir la semántica de la tarea: entregar comida, no solo encontrarla. Luego hubo que hacer aprendible el crédito: shaping de distancia, más agentes y despliegue muestreado. Después apareció el detalle más importante para 50x50: el crítico centralizado espacial. Ese cambio reestructura el aprendizaje y debe figurar como una causa mayor. La memoria por bytes sigue siendo una hipótesis interesante, no una conclusión demostrada.

La contribución honesta del trabajo no es “resolvimos comunicación emergente”. Es más precisa: construimos un entorno de forrajeo multi-agente con memoria externa, identificamos qué cambios desbloquean cada escala, alcanzamos solución robusta en 25x25 y resultados muy fuertes en 50x50 bajo crítico espacial, y mostramos que en 250x250 el shaping puede engañar si no se mide entrega real.

A. Checklist para la versión final

Antes de entregar, faltan tres pasos:

1. Elegir qué runs entran como tabla final y congelar sus checkpoints.
2. Agregar una tabla de hiperparámetros por familia, especialmente `critic_architecture`, `food_count`, `food_sources`, número de hormigas, bits y modo de evaluación.
3. Correr ablaciones matched mínimas para comunicación: no-write, byte shuffle y mismo costo entre 4 y 8 bits.

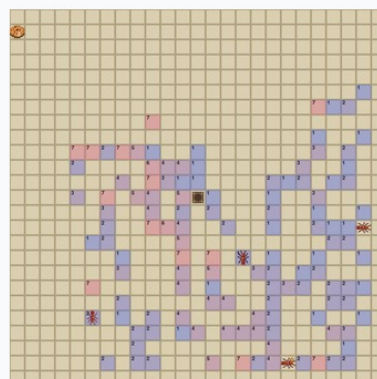
Referencias

- [1] Jakob Foerster, Ioannis Alexandros Assael, Nando de Freitas, and Shimon Whiteson. Learning to communicate with deep multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, 2016.
- [2] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems*, 2017.
- [3] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation, 2015.
- [4] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.
- [5] Sainbayar Sukhbaatar, Arthur Szlam, and Rob Fergus. Learning multiagent communication with backpropagation. In *Advances in Neural Information Processing Systems*, 2016.
- [6] Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of ppo in cooperative multi-agent games. In *Advances in Neural Information Processing Systems*, 2022.

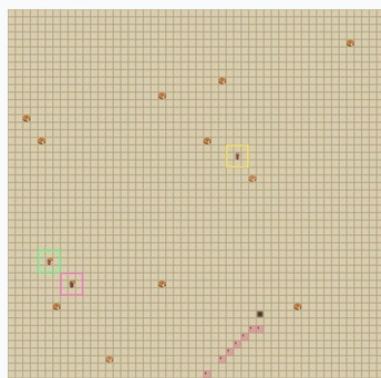
Azar: sin retorno estable



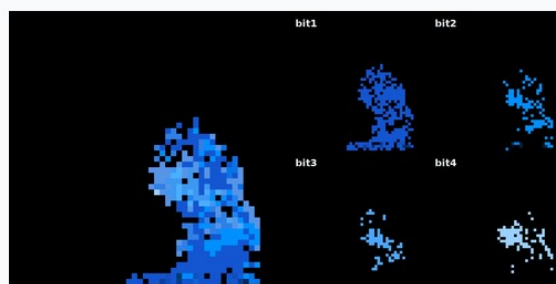
25x25: cobertura multi-hormiga



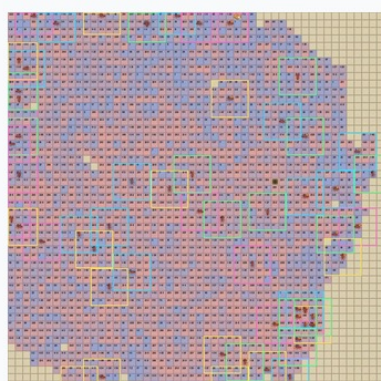
50x50 MLP: progreso parcial



50x50 strided CNN: entregas recurrentes



50x50, 60 ants: frontera reciente



250x250: reset-boundary local

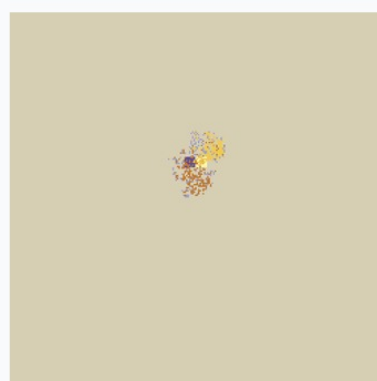


Figura 6: Fotogramas extraídos de videos locales. La figura no es evidencia cuantitativa; sirve para interpretar la semántica de los números: exploración, retorno, escala y fallas por shaping.